



西安电子科技大学
XIDIAN UNIVERSITY

广州研究院

Guangzhou Institute of
technology

● Practical Object-Level Sanitizer With Aggregated Memory Access and Custom Allocator

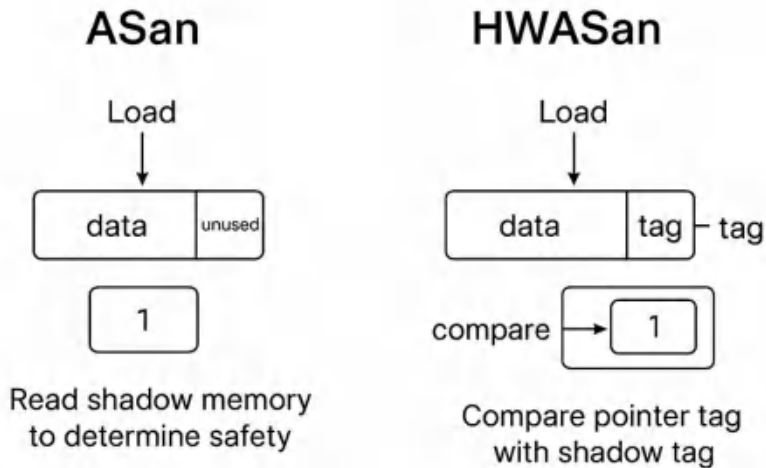
汇报人：奚佳新

时间：2025-10-23



背景——HWASan和ASan对比

传统的ASan利用读取影子内存值来判断内存访问是否安全，而HWASan利用TBI来存放对象标签，通过比较读取影子内存值中的标签和内存对象本身的标签来判断是否安全。



OLASan概述

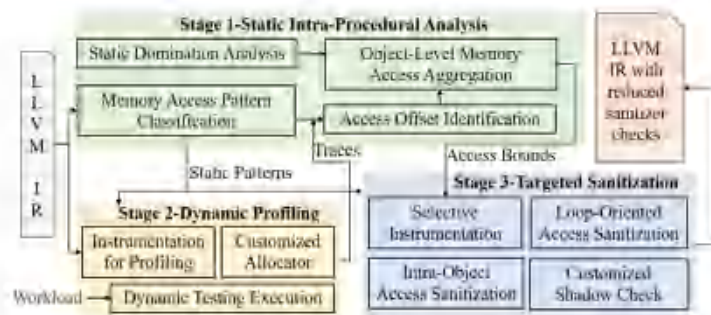


Fig. 2: System Overview

OLASan系统概述

- 静态过程内分析
对内存访问模式进行分类，合并内存访问
- 动态分析
为已分析的路径收集运行时跟踪
- 定向清理
对于已分析的路径，OLASan利用全面的有针对性的清理；对于未分析的路径，使用已有的检查比如ASan检查。



OLASan设计——静态过程内分析

1.内存访问模式分类

Pattern	Example (from Listing 1)
Access to Array	<code>tpim = tsc[TIM]</code>
Access to Struct	<code>bzf->strm, bzf->free</code>
Access in Loop	<code>for (int i = 0; i < num(); i++) set[i] = newlast;</code>
Access using Libc	<code>memcpy(mem, newmem, newlast)</code>

2.识别基地址和偏移量

```
#define TIM 3
```

```
int **tsc;
```

```
Initialize(tsc);
```

```
int *tpim = tsc[TIM];
```

```
%1 = alloca ptr, align 8
%6 = load ptr, ptr %1, align 8
%7 = getelementptr inbounds ptr, ptr %6, i64 3
%8 = load ptr, ptr %7, align 8
store ptr %8, ptr %2, align 8
```

太理想!!!

通过动态分析补充对象基址和偏移量

3.基于支配分析的内存访问聚合

Algorithm 1 Memory Access Aggregation

```
Input: All Memory Accesses  $M$  with their  $f()$  and Offset:  $S$  is  $F$ 
Output: Summed Memory Accesses  $M'$ 
1: procedure ACCESS_AGGREGATION( $M, O, S$ )
2:   Inboundness:  $M'_0 = M[0]$  if key is object base and value is address set
3:   for  $m \in M[1:n]$  do // First step
4:      $m_{base} = f(m)$ 
5:      $M'_0 = \text{approx}(M'_0, m)$ 
6:   for all  $(m, m_{base}) \in M$  do // Second step
7:     Initialization:  $groups = \{O(x) \mid O \text{ is memory for the each base}\}$ 
8:     for  $u \in \text{access\_set}$  do
9:        $group\_found = \text{False}$ 
10:      for each  $u_f = \text{getInbound}(u)$  do // Get the inbound ID being accessed
11:        for  $g \in groups$  do
12:          dominance =  $\text{False}$ 
13:          store_inbound =  $\text{True}$ 
14:          for each  $access \in g$  do
15:            if  $\text{hasOffset}(access.f(), m_{base}, u_f)$  then
16:               $store\_inbound = \text{True}$  break;
17:            if  $\text{dom}(access.f(), m_{base}, u_f) \leq \text{prev\_dom}(m_{base}, m_{base})$  then
18:              dominance =  $\text{True}$  break;
19:            if  $\text{dom}(access.f(), m_{base}, u_f) > \text{prev\_dom}(m_{base}, m_{base})$  then
20:              Add  $u$  to  $g$ 
21:          if  $\text{dom}(access.f(), m_{base}, u_f) > \text{prev\_dom}(m_{base}, m_{base})$  then
22:             $group\_found = \text{True}$  break;
23:          if  $\text{store\_inbound}$  then
24:             $groups = \text{groups} \cup \{g\}$ 
25:   for  $g \in groups$  do // Third Step
26:     if  $g$  size  $\leq 2$  then
27:       Add  $g$  to  $M'$ 
28:     else
29:        $m_{base} = \text{dom}(m_{base}, m_{base})$ 
30:       Add  $m_{base}$  to  $M'$ 
31:   return  $M'$ 
```

```
typedef struct {
    int a[4]; // offset 0 ~ 15
    int x;    // offset 16 ~ 19
    int y;    // offset 20 ~ 23
} Node;

Node *p = malloc(sizeof(Node)); // 基地址假设为 0x4f000000

void foo(int i) {
    p->a[0] = 1; // 访问 offset 0
    p->a[3] = 2; // 访问 offset 12
    if (i > 0) {
        p->x = 3; // 访问 offset 16
    } else {
        p->y = 4; // 访问 offset 20
    }
}
```

```
Mo = {
    0x4f000000: [p->a[0], p->a[3], p->x, p->y]
}
```

对 0x4f000000 的 4 条访问逐一尝试放入现有组:

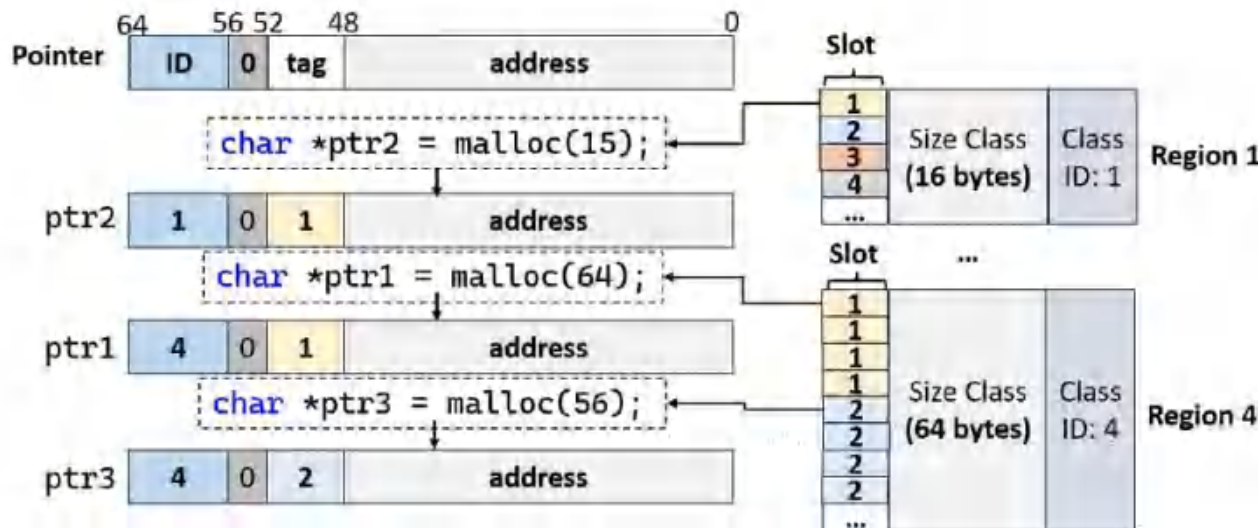
1. $p \rightarrow a[0]$ → 新建组 $g0 = \{a[0]\}$
2. $p \rightarrow a[3]$
 - 与 $g0$ 同属 entry 分支
 - 且被 $g0$ 支配 (同一基本块)
 - 加入 $g0$ → $g0 = \{a[0], a[3]\}$
3. $p \rightarrow x$
 - 分支 ID = branch-A
 - $g0$ 分支是 entry $\neq A$
 - 不能加入 $g0$, 新建组 $g1 = \{x\}$
4. $p \rightarrow y$
 - 分支 ID = branch-B
 - 与 $g0$ 、 $g1$ 分支都不同
 - 新建组 $g2 = \{y\}$

```
groups = [
    /* g0 */ [p->a[0], p->a[3]], // entry 分支, 偏移 0 与 12
    /* g1 */ [p->x],           // branch-A, 偏移 16
    /* g2 */ [p->y],           // branch-B, 偏移 20
]
```

访问指令	对象基地址 O[i]	偏移量 S[i]	所属分支 ID
$p \rightarrow a[0]$	0x4f000000	0	entry
$p \rightarrow a[3]$	0x4f000000	12	entry
$p \rightarrow x$	0x4f000000	16	branch-A
$p \rightarrow y$	0x4f000000	20	branch-B



OLASan设计——动态分析



```
// assuming: tagged 'ptr' is target allocation
class_id = ptr >> 56;
ptr_raw = ptr & 0xFFFFFFFFFFFF;
region_base = GetRegionBeginBySizeClass(class_id);
class_size = ClassIdToSize[class_id];
base = region_base + ((ptr_raw - region_base) /
    class_size) * class_size;
offset = ptr_raw - base;
```



OLASan设计——定向清理

```
bool loopStart = false; +
uintptr_t InitialAddress = 0; +
uintptr_t EndAddress = 0; +
for (uint i = 0; i < num(); i++) {
    if (!loopStart) {
        InitialAddress = set[i]; +
        loopStart = true; +
        set[i] = newlast;
        EndAddress = set[i]; +
    }
    SanitizeCheck(InitialAddress, EndAddress) +
}
```

- 选择性插桩（数组边界）
- 循环优化
- 针对 Libc 定制阴影检查
- 优化结构体

```
memcpy(cs->bitcounter, currMB->bitcounter, 3960);
// optimize sanitization for libc call
class_id = cs->bitcounter >> 56;
ptr_tag = (cs->bitcounter >> 48) && 0x000F;
ptr_raw = cs->bitcounter && 0xFFFFFFFFFFFFFFFF;
class_size = ClassIdToSize[class_id];
if (class_size < 3960) {
    ErrorAlert();
}
tag_first = getShadow(ptr_raw); //memory tag
if (tag_first != ptr_tag)
{
    ErrorAlert();
}
tag_last = getShadow(ptr_raw + 3960);
if (tag_last != ptr_tag) {
    if (!PossiblyShortTagMatchesCheck())
        ErrorAlert();
}
```



实验结果

- *RQ1: Does OLASan have a performance advantage?*
- *RQ2: Can OLASan effectively detect real bugs?*
- *RQ3: How do individual customized components affect time overhead?*
- RQ4: What are the characteristics of OLASan's profiling based removed checks through memory access aggregation?

- RQ1: Does OLASan have a performance advantage?

	Benchmark	ASan	ASan++	GrantSan	OLASan	OLASan++ ^b
SPEC2006	401.bzip2	198.18%	152.24%	RE ^a	200.99%	225.32%
	445.gobmk	192.23%	132.43%	163.5%	183.59%	197.00%
	450.soplex	197.27%	236.12%	212.58%	170.67%	203.42%
	453.povray	230.42%	279.79%	224.29%	235.29%	275.95%
	456.hmmer	200.28%	218.68%	222.81%	204.99%	223.16%
	458.sjeng	208.05%	171.5%	165.64%	190.04%	212.78%
	470.lbm	204.27%	142.6%	200.37%	169.58%	206.78%
	471.omnibpp	199.79%	RE ^a	221.32%	167.59%	252.29%
	473.astar	218.78%	249.57%	142.86%	154.62%	209.86%
SPEC2017	500.perlbench_r	238.24%	225.46%	200.56%	264.7%	277.3%
	502.gcc_r	399.25%	292.5%	278.91%	230.96%	295.63%
	505.mcf_r	222.04%	141.42%	127.82%	130.98%	165.15%
	508.namd_r	231.38%	168.23%	107.46%	133.05%	192.98%
	520.omnibpp_r	292.92%	258.94%	196.79%	130.81%	238.99%
	531.leapsjeng_r	206.45%	162.4%	141.18%	128.59%	176.45%
	538.imagick_r	196.56%	176.8%	136.47%	115.71%	187.29%
	541.leela_r	209.11%	186.2%	143.61%	135.41%	194.93%
	619.lbm_s	128.71%	116.5%	105.63%	105.03%	134.63%
	Average	220.77%	194.79%	176.11%	169.59%	214.99%

- RQ2: Can OLASan effectively detect real bugs?

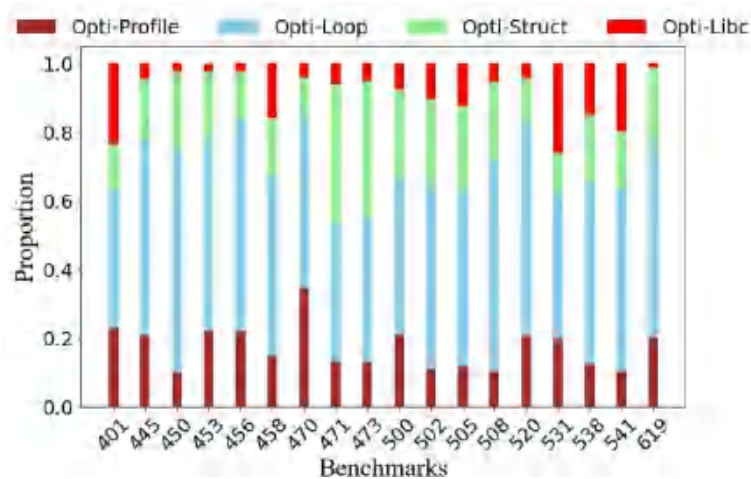
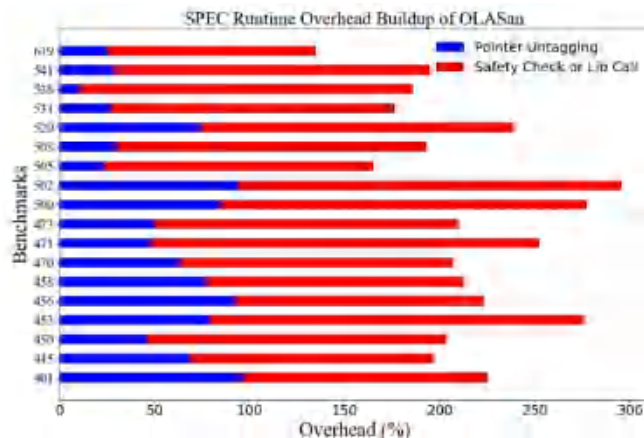
CVE	Type	Program	ASan	GiantSan	ShadowBound	OLASan
CVE-2006-6563	HBO ^a	proftpd	✓	✓	✓	✓
CVE-2009-2285	HBO	libtiff	✓	✓	✓	✓
CVE-2013-4243	HBO	libtiff	✓	✓	✓	✓
CVE-2013-7443	HBO	sqlite	✓	✓	✓	✓
CVE-2014-1912	HBO	python	✓	✓	✓	✓
CVE-2015-8668	HBO	libtiff	✓	✓	✓	✓
CVE-2015-9101	HBO	lame	✗	✗	✓	✓
CVE-2016-1762	HBO	libxml	✓	✗	✓	✓
CVE-2016-1838	HBO	libxml	✓	✓	✓	✓
CVE-2016-10095	SBO	libtiff	✓	✓	✗	✓
CVE-2016-10270	HBO	libtiff	✓	✗	✓	✓
CVE-2016-10271	HBO	libtiff	✓	✗	✓	✓
CVE-2017-7263	HBO	potrace	✓	✓	✓	✓
CVE-2017-14409	SBO ^b	mp3gain	✓	✓	✗	✓
CVE-2018-20330	HBO	libjpeg-turbo	✓	✓	✓	✓
CVE-2019-7310	HBO	poppler	✓	✓	✓	✓
CVE-2019-9021	HBO	php	✓	✓	✓	✓
CVE-2019-9200	HBO	poppler	✓	✓	✓	✓
CVE-2019-10872	HBO	poppler	✓	✓	✓	✓
CVE-2020-19131	HBO	libtiff	✓	✓	✓	✓
CVE-2020-19144	HBO	libtiff	✓	✓	✓	✓
CVE-2021-3156	HBO	sudo	✗	✗	✓	✓
CVE-2021-4214	HBO	libpng	✗	✗	✓	✓
CVE-2021-26259	HBO	hmsdoc	✓	✓	✓	✓
CVE-2021-32281	HBO	gravity	✓	✓	✓	✓
CVE-2022-0080	HBO	mruby	✗	✗	✓	✓
CVE-2022-0891	HBO	libtiff	✓	✗	✓	✓
CVE-2022-0924	HBO	libtiff	✗	✗	✓	✓
CVE-2022-28966	HBO	wasn3	✓	✓	✓	✓
CVE-2022-31627	HBO	php	✓	✓	✓	✓

CWE Name	Vulnerability Type	Number of Samples
CWE121	Stack Buffer Overflow	4896
CWE122	Heap Buffer Overflow	3777
CWE124	Buffer Underwrite	1440
CWE126	Buffer Overread	2004
CWE127	Buffer Underread	2000
CWE415	Double Free	818
CWE416	Use After Free	393
CWE761	Invalid Free	424
Total	—	15752

Name (#cases)	OLASan (15752)	GiantSan (15752)	PACMem (11531)	CryptSan (5364)	HWASan (5364)	ASan (15752)	SoftB/CETS (3970)
CWE121	100%	82.14%	98.82%	98.5%	82.9%	83.74%	77.7%
CWE122	100%	83.92%	99.07%	97.4%	94.6%	83.92%	73.7%
CWE124	100%	80.18%	100%	100%	81.9%	80.18%	82.5%
CWE126	100%	82.89%	100%	100%	99.7%	82.89%	96.5%
CWE127	100%	91.01%	100%	100%	73.9%	91.01%	78.4%
CWE415	100%	100%	100%	100%	100%	100%	100%
CWE416	100%	90.41%	100%	100%	50.9%	90.41%	51.3%
CWE761	100%	91.56%	100%	100%	0%	91.56%	100%

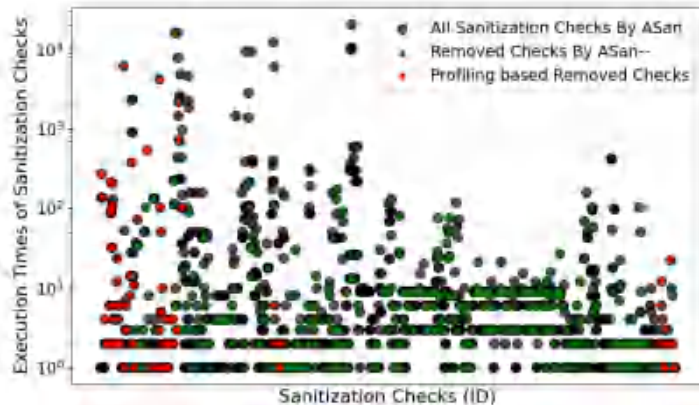
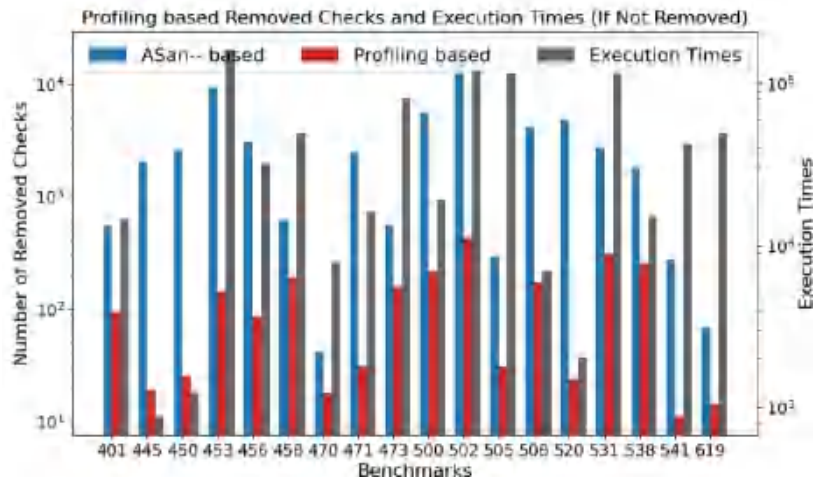
- ASan无法检测到跳过红区的跨对象溢出，或访问已释放一段时间的对象
- 由于缺乏对wcsncpy、wmemset等的检测，ASan未能检测到CWE 122中的一些堆缓冲区溢出
- ASan用来设置红区的函数asan_alloc_poison在某些情况下不起作用，导致漏报

- RQ3: How do individual customized components affect time overhead?



观察到，对于大多数基准测试，大部分开销归因于安全检查，这些检查也可能调用库函数，*OLASan*中提出的有针对性的清理技术共同促成了性能开销的显著降低，验证方法的有效性。

- RQ4: What are the characteristics of OLASan's profiling based removed checks through memory access aggregation?



OLASan虽然比ASan--移除更少的检查点，但这些检查点都集中在运行频率极高的“热路径”上，因此能在保证安全的前提下，更精准地降低性能开销。